

Trabajo Práctico 4 — Un nivel que se juega: física y monedas

Diplomatura de Videojuegos · Clase 4 Objetivo: armar un **mini plataformero** de verdad. Vas a usar los **tres cuerpos físicos** de Godot, mover al personaje con `move_and_slide()`, hacer que **choque** con el suelo gracias a `CollisionShape2D`, y que al tocar una **moneda** la recoja usando **señales** y **grupos**.

Qué vas a lograr

Un nivel jugable donde el personaje:

- **camina y salta** con física real (cae por gravedad, pisa el suelo),
- **choca** con el suelo en vez de atravesarlo,
- **junta monedas** al tocarlas, y suma puntos en la consola,
- y vas a ver, en vivo, la diferencia entre `StaticBody2D`, `CharacterBody2D` y `RigidBody2D`.

 **Tiempo estimado:** 70–100 min. Como siempre: **escribí el código vos**. Se aprende tecleando y probando.

 **Viene de la Clase 3:** aquel personaje se movía con `position += ... * delta`. Hoy el movimiento pasa por el **motor de física**, así el personaje **choca** con el mundo.

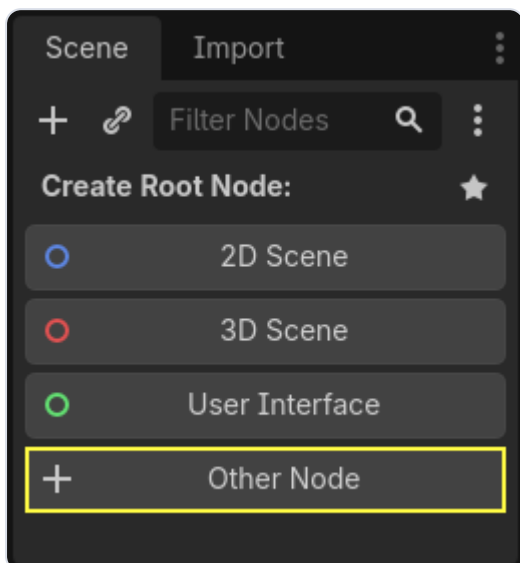
Cómo va a quedar el árbol de nodos

```
Nivel (Node2D)           ← raíz del nivel
├─ Jugador (CharacterBody2D) ← lo controlás vos
│  ├─ Sprite2D
│  └─ CollisionShape2D
├─ Suelo (StaticBody2D)   ← fijo, no se mueve
│  └─ CollisionShape2D
└─ Moneda (Area2D)        ← detecta, no bloquea
   ├─ Sprite2D
   └─ CollisionShape2D
```

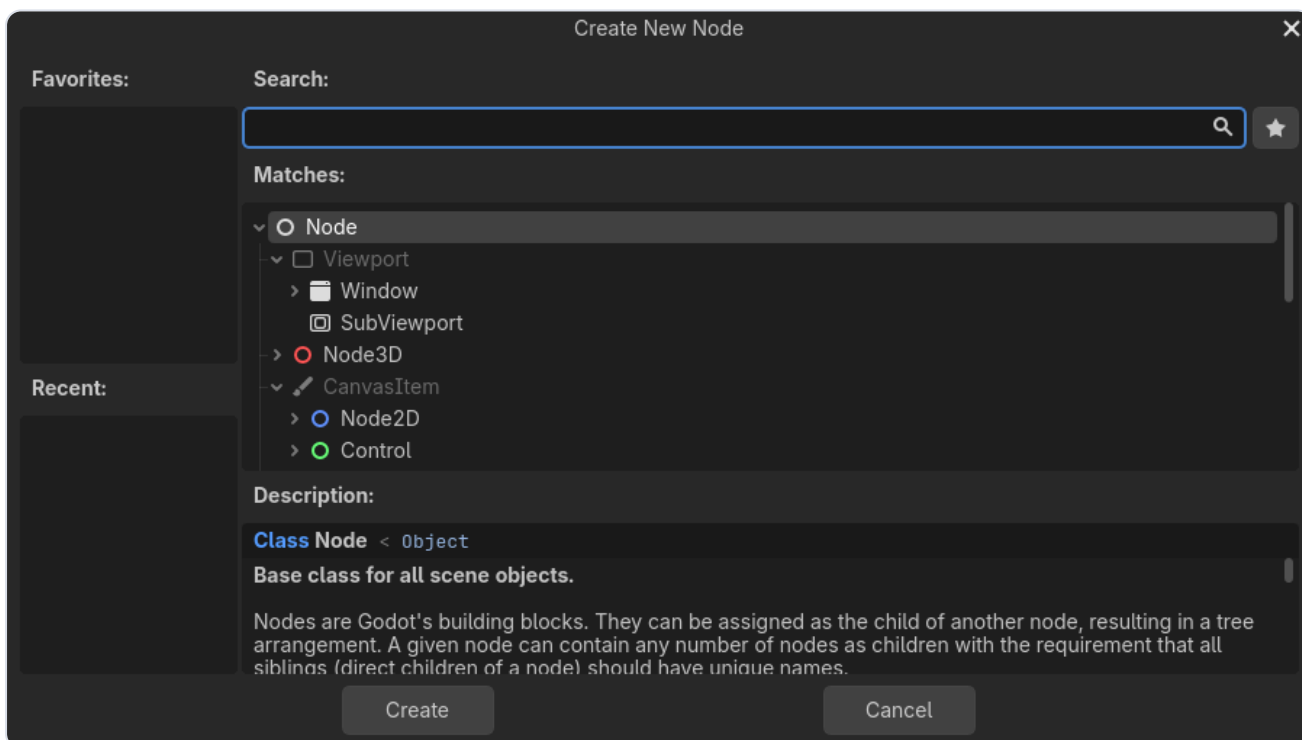
Tres nodos, tres **cuerpos físicos distintos**. Elegir el correcto es la mitad del trabajo.

Parte 0 — Proyecto y personaje

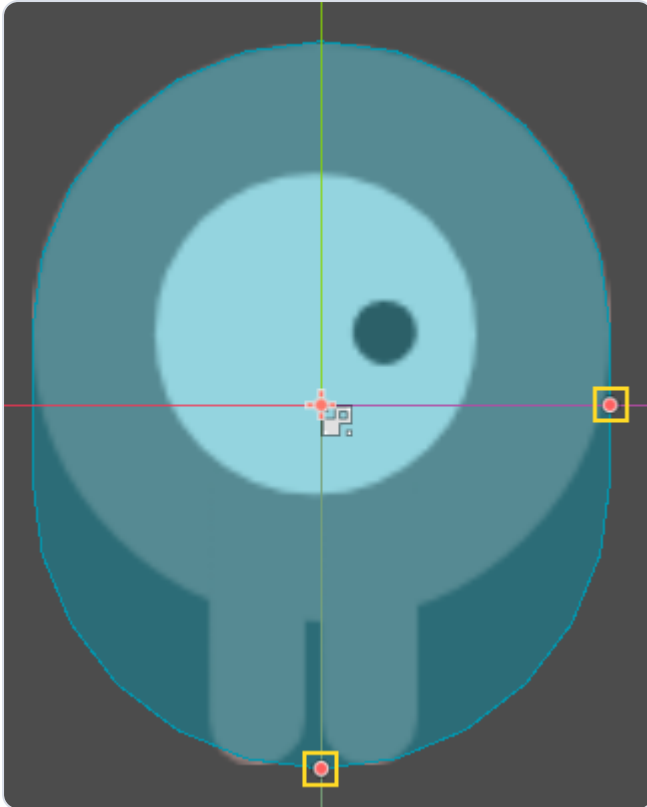
1. Abrió Godot → **New Project** → nombre `tp4-nivel` → carpeta vacía → **Create & Edit**.
2. En el panel **Escena**, clic en **Otro Nodo** (*Other Node*).



3. Buscá `Node2D`, crealo y renombralo a `Nivel`. Va a ser la **raíz** del nivel. Guardá con **Ctrl + S** → `nivel.tscn`.
4. Seleccioná `Nivel` → agregá hijo (**Ctrl + A**) → `CharacterBody2D` → renombralo `Jugador`.



5. **Darle imagen:** con `Jugador` seleccionado, agregá hijo → `Sprite2D`. Arrastrá el `icon.svg` del **FileSystem** a la propiedad **Texture**.
6. **Darle cuerpo:** seleccioná `Jugador` → hijo → `CollisionShape2D`. En el Inspector → **Shape** → **Nuevo RectangleShape2D** y ajustá los puntos naranjas para cubrir el sprite.



7. Poné al **Jugador** **arriba en la pantalla** (Inspector → **Transform** → **Position**, por ejemplo `y = 100`), así después lo vemos caer.

8. **Adjuntá el script:** clic derecho sobre **Jugador** → **Attach Script** → `res://jugador.gd` → **Create**.
Borrá la plantilla y dejá:

```
extends CharacterBody2D

func _ready():
    print("=== Jugador listo ===")
```

9. Guardá y ejecutá con **F6**. En **Output** aparece el mensaje y en la ventana, el personaje.

```
Godot Engine v4.6.rc1.official
D3D12 12_0 - Forward+ - Using
1060 66B

Hello, world!
--- Debugging process stopped
```

✓ **Punto de control 0:** ves el personaje en pantalla y el mensaje en Output. (Todavía no se mueve ni cae — eso viene ahora.)

Parte 1 — Los tres cuerpos físicos

Concepto: cuándo usar `StaticBody2D`, `CharacterBody2D` y `RigidBody2D`.

Godot tiene **tres** nodos para objetos físicos. Elegir el correcto es fundamental:

NODO	¿QUIÉN LO MUEVE?	PARA QUÉ
<code>StaticBody2D</code>	Nadie — está fijo	Suelo, paredes, plataformas
<code>CharacterBody2D</code>	Vos , por código	El jugador, enemigos con IA
<code>RigidBody2D</code>	El motor (física)	Cajas, barriles, cosas que se tiran

 **La regla práctica:** el **jugador** siempre es `CharacterBody2D`; el **suelo** siempre es `StaticBody2D`; lo que **se tira o rebota** es `RigidBody2D`.

 **En juegos que conocés:** en *Celeste* conviven los tres a la vez — Madeline (`CharacterBody2D`), las plataformas (`StaticBody2D`) y los bloques que caen (`RigidBody2D`). Lo mismo el suelo de *Mario* (static), el propio Mario (character) y los caparazones que pateás (rigid).

Elegimos `CharacterBody2D` para el `Jugador` porque queremos **control preciso**: nada de rebotes ni physics locas, se mueve **exactamente** como le decimos. Ya lo vas a sentir en la Parte 3.

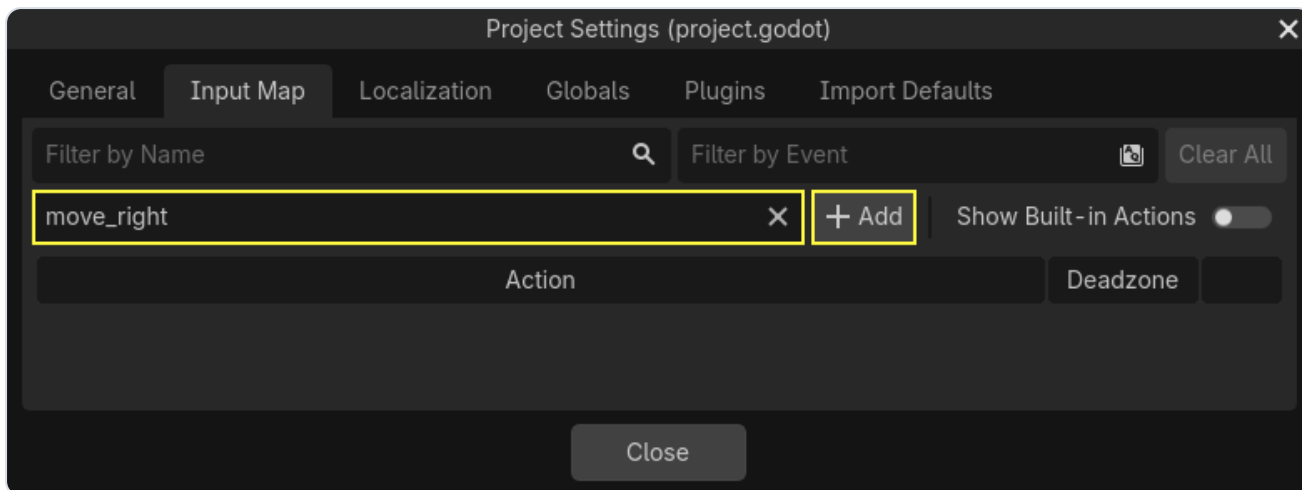
✓ **Punto de control 1:** podés explicar por qué el jugador es `CharacterBody2D` y el suelo va a ser `StaticBody2D`.

Parte 2 — El Input Map: mover y saltar

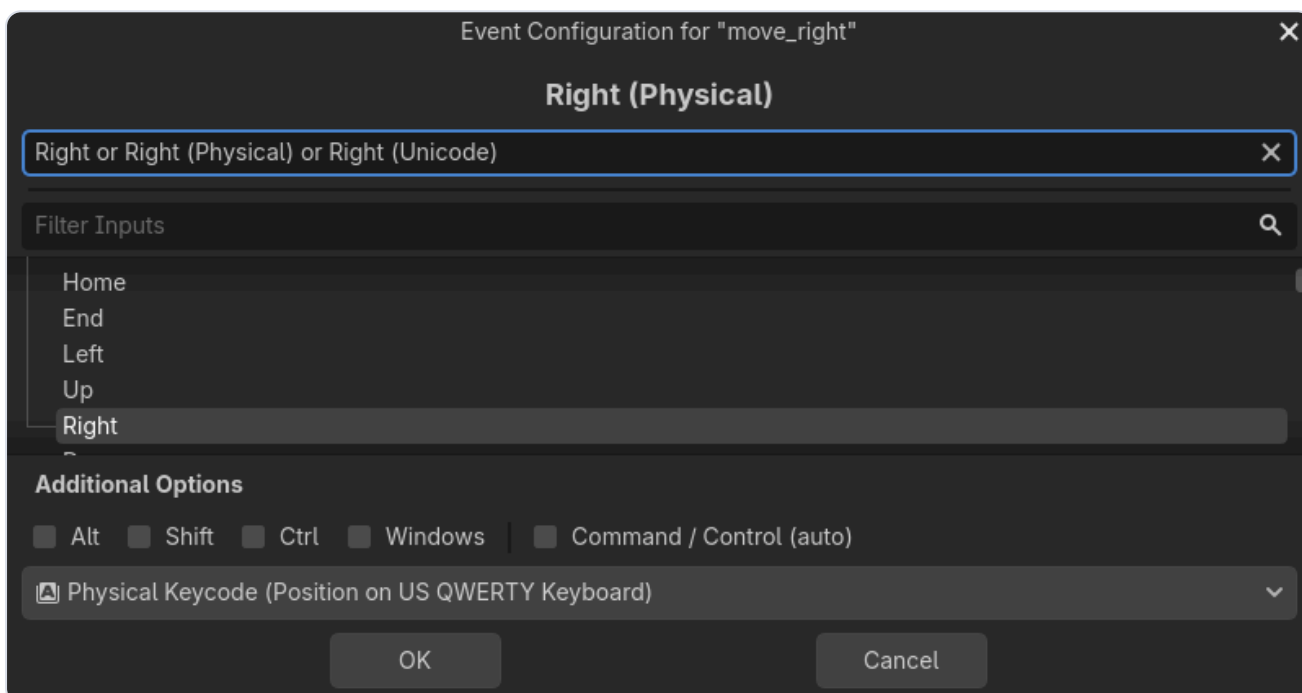
Concepto: acciones con nombre en el Input Map (como en la Clase 3).

Abrí **Project** → **Project Settings** → **Input Map** (*Proyecto* → *Configuración del proyecto* → *Mapa de Entrada*).

1. Escribí `mover_izquierda` en la barra de arriba y clic en **Add**.

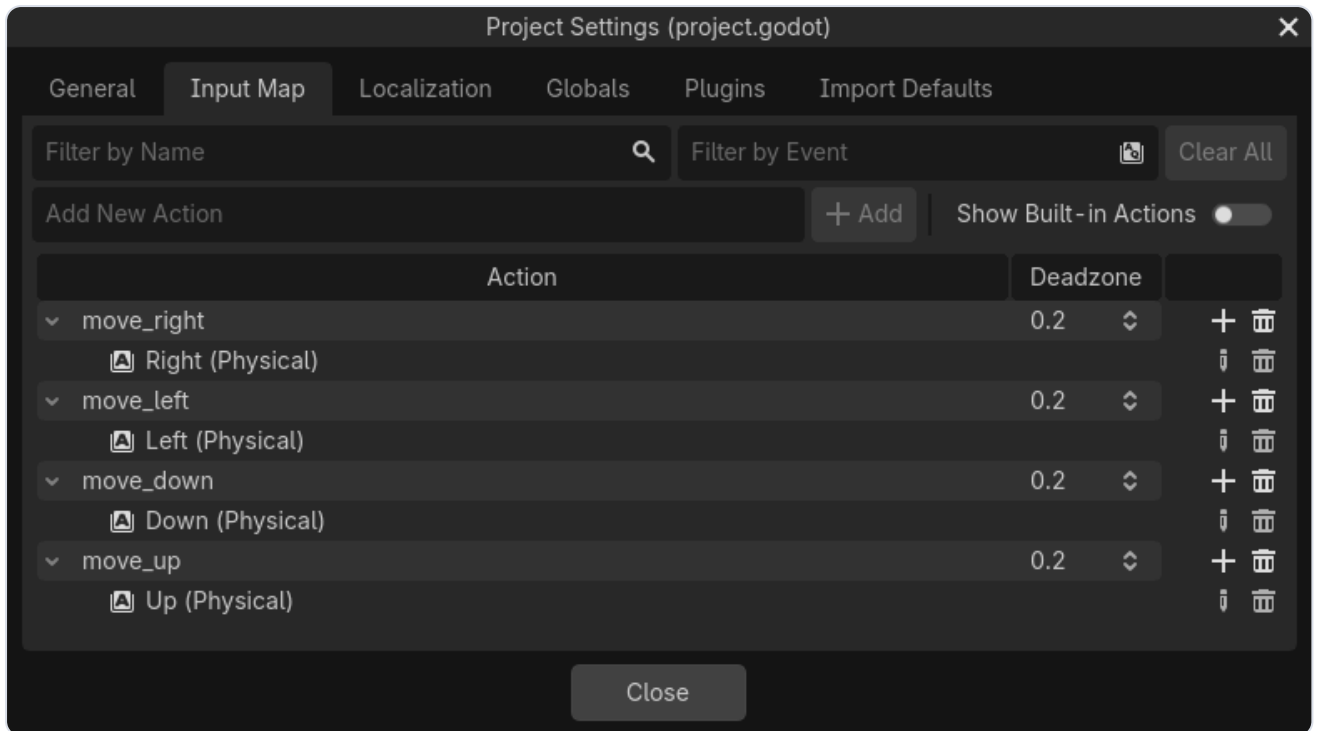


2. Clic en el  de esa acción, **presioná la tecla** (por ejemplo **A** o **←**) y **OK**.



3. Repetí para las otras dos:

ACCIÓN	TECLA SUGERIDA
<code>mover_izquierda</code>	A o ←
<code>mover_derecha</code>	D o →
<code>saltar</code>	Espacio



📁 En la captura oficial las acciones se llaman `move_*`. Vos usá los nombres de arriba; lo único que importa es que **coincidan** con el código.

✅ **Punto de control 2:** tenés `mover_izquierda`, `mover_derecha` y `saltar` en el Input Map.



Parte 3 — Movimiento con `move_and_slide()`

Conceptos: el patrón de plataformas, `velocity`, gravedad, salto, y `_physics_process()`.

En la Clase 3 movíamos con `position += ...`. Eso **no choca** con nada. Ahora usamos `move_and_slide()`: mueve al personaje con su variable `velocity`, **detecta colisiones** y ajusta el movimiento solo.

Reemplazá el script por esto:

```

extends CharacterBody2D

const VELOCIDAD = 250.0
const GRAVEDAD = 980.0
const SALTO = -450.0          # negativo = hacia arriba (Y invertido)

func _physics_process(delta):
    # 1. Gravedad: se acumula mientras esté en el aire
    if not is_on_floor():
        velocity.y += GRAVEDAD * delta

    # 2. Input horizontal: -1 (izq), 0, 1 (der)
    var dir = Input.get_axis("mover_izquierda", "mover_derecha")
    velocity.x = dir * VELOCIDAD

    # 3. Salto: solo si está tocando el suelo
    if Input.is_action_just_pressed("saltar") and is_on_floor():
        velocity.y = SALTO

    # 4. Que Godot resuelva las colisiones
    move_and_slide()

```

Apreté **F6**. El personaje **cae y se va de la pantalla** 😊 — ¡todavía no hay suelo! Es lo esperado. Cerró el juego.

🗨 Cuatro pasos, siempre los mismos:

1. **Gravedad** — `velocity.y += GRAVEDAD * delta` mientras `not is_on_floor()` .
2. **Input** — `get_axis()` da un número entre `-1` y `1` ; por la velocidad, es la dirección.
3. **Salto** — un impulso hacia arriba (`velocity.y` **negativo**), solo si está en el suelo.
4. **Mover** — `move_and_slide()` aplica `velocity` , resuelve choques y actualiza `is_on_floor()` .

⚠ **Va en `_physics_process()` , no en `_process()` .** Todo lo que usa **física y colisiones** va acá: se llama a **frecuencia fija** (60 Hz), estable en cualquier máquina. (Por eso acá el `delta` ya viene "parejo".)

✅ **Punto de control 3:** el personaje cae por gravedad (y se va de pantalla, porque falta el suelo).

Parte 4 — El suelo (StaticBody2D)

Concepto: un `StaticBody2D` con su `CollisionShape2D` para que el personaje aterrice.

1. Seleccioná `Nivel` → agregá hijo → `StaticBody2D` → renombralo `Suelo`.
2. Con `Suelo` seleccionado, agregá hijo → `CollisionShape2D` → **Shape** → **Nuevo RectangleShape2D**.
3. En el Viewport, **estirá** el rectángulo hasta que sea una **plataforma ancha**, y posicioná el `Suelo` **abajo** (Inspector → Position, por ejemplo `y = 400`).
4. (Opcional) Agregale un `Sprite2D` para verlo, o dejalo invisible: la colisión funciona igual.
5. Apretá **F6**. Ahora el personaje **cae y aterriza** sobre el suelo, camina con A/D y salta con Espacio. 🎮

🗨 **Por qué el suelo es `StaticBody2D`** : no se mueve nunca, pero **frena** al personaje. Es la pareja perfecta del `CharacterBody2D` : uno empuja, el otro lo detiene. Fijate que **no escribiste ni una línea** para la colisión — la resuelve `move_and_slide()`.

🧪 **Experimento (los 3 cuerpos juntos):** agregá al `Nivel` un `RigidBody2D` con su `Sprite2D` + `CollisionShape2D`, ponelo arriba y ejecutá. La caja **cae sola y se apoya** en el suelo (física del motor), mientras el personaje **la controlás vos**. Ahí ves los tres tipos conviviendo: static (suelo), character (jugador), rigid (caja). Después podés borrarla.

✅ **Punto de control 4:** el personaje aterriza en el suelo, camina y salta.

🔧 El personaje atraviesa el suelo o no se mueve

▼ Abrí para ver soluciones

- ¿El `Suelo` y el `Jugador` tienen cada uno su `CollisionShape2D` con **shape asignado**? Sin shape, no chocan.
- ¿Hiciste **click en la ventana del juego** para darle foco al teclado?
- Los nombres de las acciones deben ser **idénticos** a los del Input Map.
- El movimiento va en `_physics_process(delta)`.



Parte 5 — Señales y `Area2D` : la moneda

Conceptos: qué son las señales y cómo `Area2D` detecta con `body_entered`.

Una **señal** es el sistema de comunicación de Godot: un nodo **avisa** que pasó algo, sin conocer a quién le avisa. Es como una **campana** — la moneda la toca cuando la agarran, y quien esté escuchando reacciona.

Area2D es un nodo que **detecta** cuándo otro cuerpo entra a una zona, pero **no lo bloquea** (a diferencia del suelo). Perfecto para monedas, poderes y zonas de daño.

Armar la moneda

1. Seleccioná **Nivel** → agregá hijo → **Area2D** → renombralo **Moneda**.
2. Hijo de **Moneda** → **Sprite2D** → arrastrale el **icon.svg**, y bajale la escala (por ejemplo **0.3, 0.3**).
3. Hijo de **Moneda** → **CollisionShape2D** → **Shape** → **Nuevo CircleShape2D** (una moneda es redonda). Ajustá el radio al sprite.
4. Posicioná la **Moneda** **sobre el suelo pero separada del jugador**, para que la toque al caminar.

Conectar la señal (por código)

Adjuntá un script a **Moneda** (**res://moneda.gd**) con esto:

```
extends Area2D

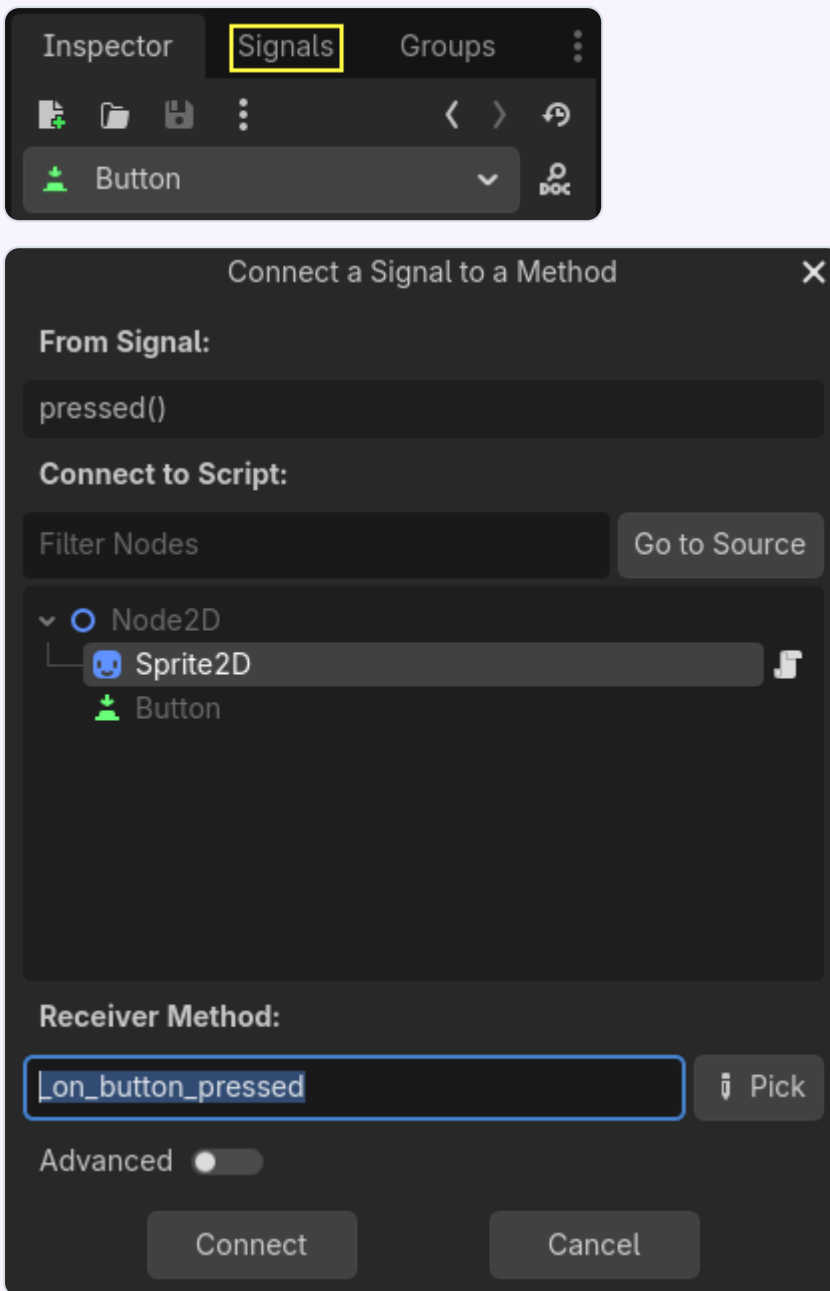
func _ready():
    # Cuando un cuerpo entra al área, llamá a _on_body_entered
    body_entered.connect(_on_body_entered)

func _on_body_entered(body):
    print("Algo tocó la moneda: " + body.name)
```

Apretá **F6** y caminá hasta la moneda: en **Output** aparece "Algo tocó la moneda: Jugador".

🗨 **El flujo de una señal:** la **Area2D** **emite** **body_entered** cuando algo entra → nosotros la **conectamos** a nuestra función **_on_body_entered** → cuando se emite, la función **se ejecuta sola**. La moneda no sabe *quién* entró de antemano; se entera cuando pasa.

🎓 **También se puede desde el editor** (sin código): selecciona la `Area2D`, anda a la pestaña **Signals** (al lado del Inspector), doble clic en `body_entered` y **Connect**. Godot te crea la función sola.



⚠️ **Si la señal no dispara:** la `Area2D` necesita un `CollisionShape2D` hijo **con shape**. Sin forma, no tiene volumen y no detecta nada.

✅ **Punto de control 5:** al tocar la moneda, se imprime un mensaje en Output.



Parte 6 — Grupos: que sepa quién la tocó

Concepto: etiquetar nodos con **grupos** para saber con quién estamos hablando.

Ahora mismo la moneda reacciona a **cualquier** cuerpo. Queremos que reaccione **solo al jugador**. Para eso lo **etiquetamos** con un grupo.

1) En `jugador.gd`, agregá el grupo y una función para sumar puntos:

```
extends CharacterBody2D

const VELOCIDAD = 250.0
const GRAVEDAD = 980.0
const SALTO = -450.0

var puntos = 0

func _ready():
    add_to_group("jugador")                # ← etiqueta
    print("Jugador listo. Puntos: " + str(puntos))

func _physics_process(delta):
    if not is_on_floor():
        velocity.y += GRAVEDAD * delta
    var dir = Input.get_axis("mover_izquierda", "mover_derecha")
    velocity.x = dir * VELOCIDAD
    if Input.is_action_just_pressed("saltar") and is_on_floor():
        velocity.y = SALTO
    move_and_slide()

func sumar_punto():                        # ← la llama la moneda
    puntos += 1
    print("¡Moneda! Puntos: " + str(puntos))
```

2) En `moneda.gd`, chequeá el grupo y recogé la moneda:

```

extends Area2D

func _ready():
    body_entered.connect(_on_body_entered)

func _on_body_entered(body):
    if body.is_in_group("jugador"):
        body.sumar_punto()    # llamamos una función del jugador
        queue_free()         # la moneda se elimina a sí misma

```

Apretá **F6** y tocá la moneda: suma un punto y **desaparece**. 🎉

🗨 **Por qué grupos:** `is_in_group("jugador")` nos deja preguntar "¿esto es el jugador?" sin depender del **nombre** del nodo. Si mañana renombrás `Jugador` a `Heroe`, **nada se rompe**. Y `queue_free()` elimina la moneda de forma segura al final del frame.

✅ **Punto de control 6:** la moneda solo reacciona al jugador, suma un punto y se elimina.

🏆 Parte 7 — Todo junto: un nivel con varias monedas

1. Duplicá la `Moneda` (seleccionala → **Ctrl + D**) dos o tres veces y movelas a distintos lugares sobre el suelo. **Todas comparten el mismo script** — no tenés que tocar nada.
2. Que aparezca un mensaje de **victoria** al juntarlas todas. En `jugador.gd`, cambiá `sumar_punto()`:

```

func sumar_punto():
    puntos += 1
    print("¡Moneda! Puntos: " + str(puntos))
    if puntos >= 3:
        print("🏆 ¡Ganaste! Juntaste todas las monedas.")

```

3. Apretá **F6** y junta las 3 monedas: al llegar a la última, salta el mensaje de victoria.

Cómo se juega: **A/D** (o flechas) para caminar, **Espacio** para saltar, tocá las monedas para juntarlas.

✅ **Punto de control 7 (final):** un nivel jugable — caminar, saltar, juntar 3 monedas y ganar.

Errores comunes al armar el final

▼ Abrió para ver soluciones

- **La moneda no suma / da error** `sumar_punto` : la función va en `jugador.gd` , y el jugador tiene que estar en el grupo `"jugador"` (`add_to_group`).
- **Una moneda no reacciona**: revisá que tenga su `CollisionShape2D` con shape, y que esté **separada del jugador** (que la toque al moverse).
- **Suma 2 puntos de golpe**: puede pasar si la moneda arranca **encima** del jugador. Separalas.
- **Indentación mezclada**: usá espacios o tabs en todo el archivo, no los combines.

Entrega

Entregá **una** de estas opciones (según indique el/la docente):

1. Los archivos `jugador.gd` y `moneda.gd` , o
2. La **carpeta del proyecto** comprimida en `.zip` (sin la carpeta `.godot/`), o
3. Un **video corto** (o GIF) juntando las monedas y viendo el mensaje de victoria.

Nombre del archivo: `tp4-nivel-ApellidoNombre.zip`

✓ Checklist de autoevaluación

- ☐ El `Jugador` es un `CharacterBody2D` con `Sprite2D` y `CollisionShape2D` .
- ☐ El `Suelo` es un `StaticBody2D` con `CollisionShape2D` , y el personaje **atteriza** en él.
- ☐ El movimiento usa `move_and_slide()` dentro de `_physics_process()` .
- ☐ Hay **gravedad** y **salto** (solo cuando `is_on_floor()`).
- ☐ La `Moneda` es un `Area2D` con `CollisionShape2D` .
- ☐ Conectaste la señal `body_entered` (por código o desde el editor).
- ☐ El jugador está en el grupo `"jugador"` y la moneda lo verifica con `is_in_group` .
- ☐ Al juntar **3 monedas** aparece el mensaje de victoria.

Extra (opcional, para los que quieran más)

- **Zona de daño**: otra `Area2D` (con pinchos) que, al entrar el jugador, imprima "¡Ay!" y le reste vida. Mismo patrón que la moneda.

- **Game Over al caer:** un `Area2D` grande **debajo** del nivel; si el jugador entra, reaparece en la posición inicial (`position = Vector2(x, y)`).
 - **Reiniciar el nivel:** al ganar, `get_tree().reload_current_scene()` para volver a empezar.
 - **Grupos como megáfono:** poné varios enemigos en el grupo `"enemigos"` y, con una explosión, `get_tree().call_group("enemigos", "recibir_dano", 50)` les pega a todos a la vez.
 - **Contador en pantalla:** en vez de la consola, mostrará los puntos con un `Label` dentro de un `CanvasLayer` (como en el TP1).
-



Recursos

- Usar CharacterBody2D: [Using CharacterBody2D](#)
- Señales, paso a paso: [Using signals](#)
- Cuerpos físicos: [Physics introduction](#)
- Grupos: [Grupos \(Groups\)](#)

Las capturas de este documento provienen de la **documentación oficial de Godot Engine** (Juan Linietsky, Ariel Manzur y la comunidad), bajo licencia **CC BY 4.0**.